# NetSage AI - Project Summary

AI-assisted Network Troubleshooting & Configuration Diagnostic Assistant

## 1. Project Information

| | |
|---|---|
| **Project Name:** | NetSage AI |
| **Project Type:** | AI-assisted network troubleshooting and configuration diagnostic assistant |
| **Creator:** | Vaibhav Amulani |
| **College:** | LNCTS |
| **Degree:** | B.Tech |
| **Branch:** | Computer Science and Engineering (CSE) |
| **Current Semester:** | 4th Semester |
| **Repository:** | https://github.com/Vaibhav2176/NetSage-AI |
| **Live Application:** | https://netsage-ai-by-vaibhav2176a.streamlit.app/ |

## 2. Project Overview

NetSage AI is a network troubleshooting assistant designed to help users understand and diagnose common networking problems, especially in Cisco IOS and Cisco Packet Tracer environments.

The main idea behind the project is to combine two different approaches to troubleshooting:

- Deterministic rule-based checking for reliable and evidence-based fault detection.
- AI-assisted diagnosis using Google Gemini for explaining the detected problem in a clearer and more useful way.

The project was designed as a decision-support system, not as an autonomous network controller. The application does not directly change router or switch configurations. Instead, it identifies the likely problem, explains why it believes the problem exists, suggests a diagnostic command and possible fix, and then leaves the final decision to a human reviewer.

This approach was chosen because network troubleshooting requires both accuracy and understandable explanations. A rule-based system is useful for detecting known configuration problems reliably, while an AI model is useful for turning those findings into a more complete diagnosis.

## 3. Problem Statement

Network troubleshooting often requires checking several pieces of information at the same time, such as:

- The reported symptom
- Network topology
- Router and switch configuration
- Cisco show command output
- VLAN and trunk configuration
- Routing information
- ACLs
- NAT
- DHCP
- OSPF and other routing protocols

- Security-related configuration

For a student or network engineer, finding the actual cause from all this information can take time. It is also easy to overlook a small configuration mistake that is responsible for the failure.

At the same time, relying only on a large language model for network diagnosis can introduce another problem: the model may produce a technically plausible answer that is not actually supported by the evidence.

NetSage AI was developed to address both sides of this problem.

The system first checks the available evidence using deterministic rules and then uses AI to explain and prioritize the result. This gives the user a diagnosis that is both grounded in the supplied evidence and easier to understand.

## 4. Proposed Solution

The solution implemented in NetSage AI follows this basic process:

```
Case / Network Evidence
        |
        v
Case Loader
        |
        v
Deterministic Rule Checker
        |
        v
Structured Findings
        |
        v
Gemini AI Diagnosis
        |
        v
Structured Diagnosis
        |
        v
Human Review
        |
        +---- Approve
        |
        +---- Edit
        |
        +---- Reject
        |
        v
Audit Log
```

The deterministic checker is always executed first.

If Gemini is available, the deterministic findings are provided to the AI as part of the case context. Gemini then produces a structured diagnosis based on the available evidence.

If Gemini is unavailable or the AI response cannot be processed correctly, the application falls back to a deterministic-only diagnosis instead of failing completely.

## 5. How I Implemented the Project

I implemented the project as a modular Python application with a Streamlit-based user interface.

The major components are separated according to their responsibilities so that the rule engine, AI layer, audit system, evaluation code and user interface can be maintained independently.

### 5.1 Case Dataset

The networking scenarios are stored in:

```
data/cases.csv
```

The dataset contains 30 network troubleshooting cases.

Each case provides the information required by the application, including the symptom, topology information, expected fault and Cisco evidence.

Keeping the cases in a separate dataset makes it easier to add or modify scenarios without duplicating the case information throughout the source code.

### 5.2 Deterministic Rule Checker

The deterministic troubleshooting logic is implemented in:

```
src/checker.py
```

The checker examines the supplied evidence for known networking fault patterns.

The project currently contains 34 deterministic troubleshooting rules, covering problems such as:

- Administratively down interfaces
- DHCP pool exhaustion
- DNS configuration problems
- OSPF timer mismatches
- ACL problems
- NAT/PAT configuration issues
- VLAN and trunking problems
- Incorrect default gateways
- DHCP relay problems
- Static routing problems
- RADIUS configuration issues
- VTP mismatches
- Dynamic ARP Inspection
- Port security violations
- HSRP timer mismatches
- Router sub-interface configuration
- IPv6 Router Advertisements
- CDP configuration

Each matching rule produces structured information rather than just returning a simple yes/no result.

### 5.3 AI Diagnosis Using Gemini

The AI part of the project is handled through:

```
src/ai_client.py
src/engine.py
prompts/diagnose_prompt.md
```

Google Gemini is used to turn the case evidence and deterministic findings into a structured diagnosis.

The AI is instructed to:

- Use only the evidence supplied by the application
- Give priority to deterministic findings
- Separate observed evidence from inference
- Identify the most likely root cause
- Identify the relevant OSI layer

- Provide a confidence score
- Give exactly one next diagnostic command
- Provide ordered fix steps
- Mention uncertainty when evidence is incomplete
- Never claim that a configuration change was actually performed
- Return only valid JSON

The diagnosis contains information such as:

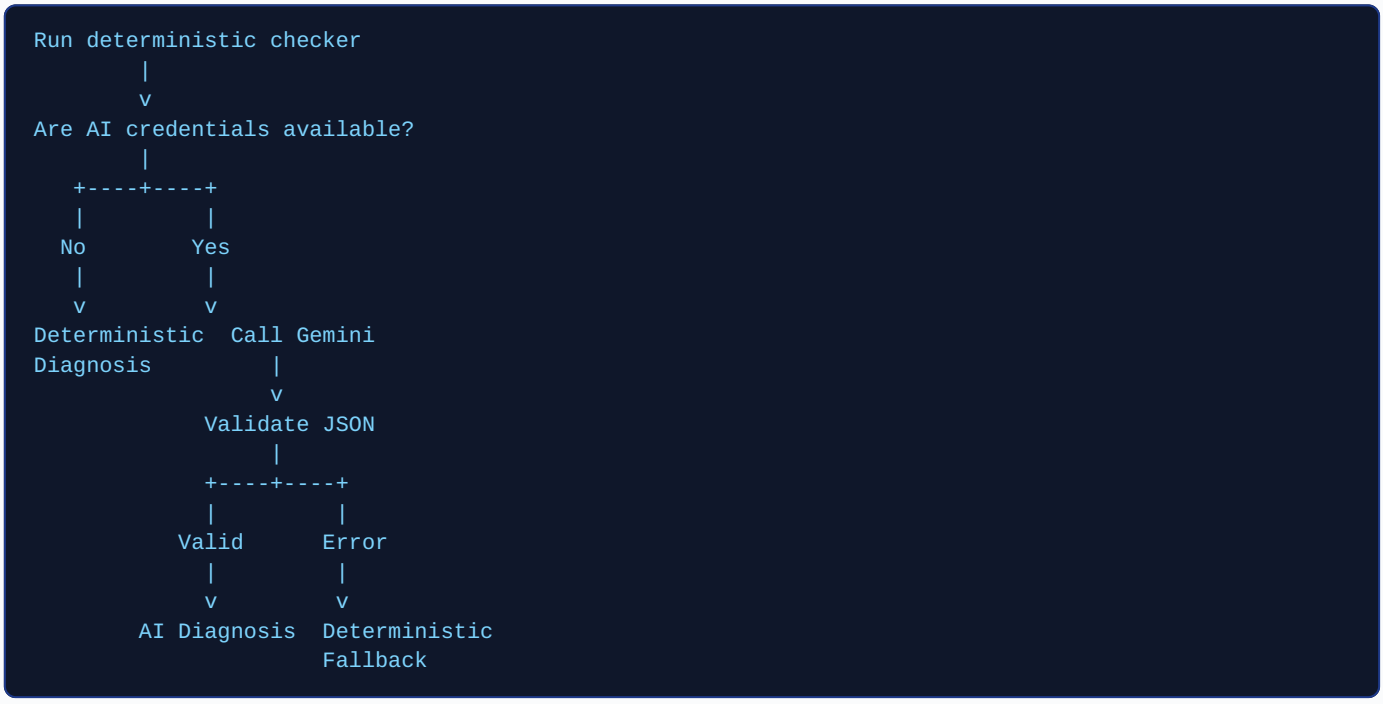
- Root cause
- OSI layer
- Confidence
- Evidence
- Next diagnostic command
- Fix steps
- Networking concept
- Severity
- Explanation
- Caveats

The structured format makes the AI output easier for the application to validate and display.

### 5.4 Deterministic-First and AI-Fallback Design

One of the main design decisions in the project is that Gemini is not treated as the only source of truth.

The application follows this logic:




This means the application remains usable even when Gemini is not configured or when an AI request fails.

This was an important part of the implementation because a network troubleshooting application should not become completely unusable just because an external AI service is temporarily unavailable.

## 6. Human-in-the-Loop Review

The final diagnosis is not automatically accepted.

The Streamlit dashboard provides a human review stage where the user can:

- **Approve**
  Accept the diagnosis as it is.
- **Edit**
  Modify the diagnosis before accepting it.
- **Reject**
  Reject the diagnosis and provide a reason.

The application does not execute Cisco commands automatically.

For example, if the system recommends:

```
interface GigabitEthernet0/0.10
no shutdown
```

the application only displays this as a recommendation. A human must decide whether the change is correct and apply it separately.

This keeps the project focused on decision support rather than autonomous network control.

## 7. Audit Logging

Human review actions are recorded in:

```
outputs/audit_log.jsonl
```

The audit system is implemented in:

```
src/audit.py
```

Each review is stored as a JSON line containing information such as:

- Timestamp
- Case ID
- Symptom
- Deterministic findings
- AI or deterministic diagnosis
- Human action
- Edited diagnosis, when applicable
- Rejection reason, when applicable
- Confidence
- Diagnosis provider

This makes it possible to look back at how the system performed and what changes a human reviewer made to its recommendations.

The audit log also provides useful information for identifying situations where the deterministic rules or AI diagnosis need improvement.

## 8. Streamlit Dashboard

The main user interface is implemented in:

```
app.py
```

The application is deployed using Streamlit.

The dashboard allows the user to:

- Select a troubleshooting scenario.
- Inspect the case and network evidence.
- Run deterministic validation.
- Generate an AI diagnosis when Gemini is configured.
- Review the root cause and supporting evidence.
- Inspect the recommended diagnostic command and fix.
- Approve, edit or reject the recommendation.
- View audit history and project statistics.

The dashboard also provides statistics such as:

- Total cases
- Reviewed cases
- Approved, edited and rejected outcomes
- Human acceptance rate
- Issue-type distribution
- Severity distribution
- Automated verification coverage
- Case-by-case coverage

## 9. Evaluation and Testing

The project includes a separate evaluation script:

```
evaluate.py
```

This compares the deterministic rule-checker results against the expected faults in all 30 cases.

**The final evaluation result is:**

- Total cases: **30**
- Correct deterministic detections: **30**
- Missed (no rule matched): **0**
- Mismatched (a rule matched, but not the expected fault): **0**
- Accuracy: **100.0%**

The project also contains automated tests under:

```
tests/
```

The final test run passed:

**✓ 31 passed**

So the deterministic troubleshooting layer achieved 30/30 correct detections on the supplied evaluation dataset, while the automated test suite achieved 31/31 passing tests.

These results are based on the project's provided 30-case dataset and should be understood in that context rather than being treated as proof that the system will correctly diagnose every possible real-world network problem.

## 10. Responsible AI Approach

Responsible use of AI was considered throughout the design of the project.

The main safeguards are:

- **Evidence grounding**
  Gemini is instructed to use only the evidence supplied by the application.
- **Deterministic-first reasoning**
  The rule checker provides grounded findings before the AI is called.
- **Human review**
  A person must review the final recommendation.
- **No automatic configuration changes**
  The application never directly executes network configuration commands.
- **Confidence and caveats**
  The AI must provide a confidence value and clearly mention missing or uncertain information.
- **Structured output**
  The AI response follows a defined JSON structure instead of returning unrestricted text.
- **Auditability**
  Human decisions and AI-influenced recommendations are recorded in the audit log.

These controls were included so that the AI layer supports the troubleshooting process without replacing human judgment.

## 11. Technologies Used

The main technologies used in the project are:

- **Python** — Main programming language
- **Streamlit** — Web dashboard and application interface
- **Google Gemini API** — AI-assisted diagnosis
- **Pandas** — Dataset and statistics handling
- **Pytest** — Automated testing
- **JSON / JSONL** — Structured AI responses and audit records
- **CSV** — Network troubleshooting case dataset
- **Excel/XLSX** — Dashboard statistics deliverable
- **Git and GitHub** — Version control and project hosting
- **Streamlit Cloud** — Application deployment

## 12. Project Documentation and Deliverables

The project includes documentation covering the system architecture, troubleshooting logic, user instructions, model auditing and AI prompt design.

The final submission deliverables are:

```
deliverables/
├── checker/
│   ├── checker.py
│   └── sample_output.txt
├── dashboard_statistics.xlsx
├── diagnose_prompt.md
└── responsible_ai_log.md
```

The project documentation is stored separately under:

```
docs/
```

This keeps the implementation, supporting documentation and submission artifacts organized.

## 13. Limitations

There are some limitations to the current version of NetSage AI.

First, the deterministic checker is based on a defined set of networking patterns. It cannot identify every possible network fault that could occur in a real environment.

Second, the AI diagnosis depends on the quality and completeness of the evidence provided to it. If important information is missing, the diagnosis may be uncertain.

Third, the current evaluation uses the project's 30 prepared cases. The reported 100% deterministic accuracy therefore represents performance on this evaluation dataset, not a guarantee of real-world accuracy across all network environments.

Finally, the application is designed as a troubleshooting and educational decision-support tool. It is not intended to replace an experienced network engineer or operate as an autonomous configuration system.

## 14. About the Creator

### Vaibhav Amulani

| | |
|---|---|
| **Name:** | Vaibhav Amulani |
| **College:** | LNCTS |
| **Degree:** | B.Tech |
| **Branch:** | Computer Science and Engineering (CSE) |
| **Current Semester:** | 4th Semester |

I am Vaibhav Amulani, a B.Tech Computer Science and Engineering student at LNCTS. NetSage AI was developed as a project to apply my interests in computer networking, software development and artificial intelligence to a practical problem.

Through this project, I worked on the networking troubleshooting logic, Python implementation, Gemini integration, Streamlit dashboard, testing, evaluation, documentation and deployment.

The project gave me an opportunity to combine networking concepts with modern AI techniques while also learning about practical aspects such as validation, fallback handling, human review and responsible use of AI.

## 15. Conclusion

NetSage AI brings together deterministic network troubleshooting and AI-assisted explanation in a single application.

The system starts with known, evidence-based troubleshooting rules and then uses Gemini to produce a structured diagnosis. The result is presented to a human reviewer, who can approve, edit or reject the recommendation. The review is then recorded for future analysis.

The final implementation contains:

- 30 networking troubleshooting cases
- 34 deterministic troubleshooting rules
- Gemini-based structured diagnosis
- Deterministic fallback when AI is unavailable
- Human-in-the-loop review
- Audit logging
- Streamlit dashboard
- Automated testing
- Deterministic evaluation

- Responsible AI documentation
- Deployment through Streamlit

The main goal of the project was not simply to add AI to a networking application. It was to build a troubleshooting workflow where AI is useful while still keeping the diagnosis grounded in evidence and the final decision with a human.